

## DAY 9 — RERANKING + CONTEXT ASSEMBLY + ENGINEERING RETRIEVAL

### GOAL

Move from:

retrieved chunks 

to:

BEST engineering context for LLM 

---

### WHY DAY 9 IS IMPORTANT

Current Day 8:

semantic retrieval 

keyword retrieval 

BUT:

results still noisy

duplicates exist

ranking weak

Example:

Average rows

Maximum rows

duplicate pages

partial matches

---

### DAY 9 FIXES

 reranking

 duplicate removal

 context assembly

 better LLM prompts

 cleaner final answers

---

## ✅ DAY 9 ARCHITECTURE

User Query

↓

Semantic Retrieval

+

Keyword Retrieval

↓

Merge Results

↓

Reranker ✅

↓

Deduplication ✅

↓

Context Builder ✅

↓

LLM Prompt ✅

---

## ✅ TEMPLATE STRUCTURE

app/

├── reranker/

| ├── rerank.py ✅

|

├── context/

| ├── context\_builder.py ✅

|

├── query/

| ├── hybrid\_search.py ✅ MODIFY

|

└─ agent/

| └─ ask.py  NEW

---

## STEP 1 — INSTALL RERANKER

pip install sentence-transformers

---

## STEP 2 — RERANKER

 **app/reranker/rerank.py**

```
from sentence_transformers import (  
    CrossEncoder  
)
```

```
reranker = CrossEncoder(  
    "BAAI/bge-reranker-base"  
)
```

```
def rerank_results(query, documents):
```

```
    pairs = []
```

```
    for doc in documents:
```

```
        pairs.append([  
            query,  
            doc["document"]  
        ])
```

```
    scores = reranker.predict(pairs)
```

```
ranked = []
```

```
for doc, score in zip(documents, scores):
```

```
    doc["rerank_score"] = float(score)
```

```
    ranked.append(doc)
```

```
ranked = sorted(
```

```
    ranked,
```

```
    key=lambda x: x["rerank_score"],
```

```
    reverse=True
```

```
)
```

```
return ranked
```

---

### ✅ STEP 3 — IMPROVE HYBRID SEARCH

 **app/query/hybrid\_search.py**

```
from app.query.semantic_search import (
```

```
    semantic_search
```

```
)
```

```
from app.query.keyword_search import (
```

```
    keyword_search
```

```
)
```

```
from app.reranker.rerank import (
```

```
    rerank_results
```

```
)
```

```
def hybrid_search(query):

    semantic = semantic_search(
        query,
        top_k=10
    )

    semantic_docs = []

    docs = semantic["documents"][0]
    metas = semantic["metadatas"][0]

    for doc, meta in zip(docs, metas):

        semantic_docs.append({
            "document": doc,
            "metadata": meta,
            "source": "semantic"
        })

    keyword_docs = keyword_search(query)

    combined = semantic_docs + keyword_docs

    # ✅ deduplicate
    unique = {}

    for item in combined:
```

```
unique[item["document"]] = item
```

```
final_docs = list(unique.values())
```

```
#  rerank
```

```
reranked = rerank_results(
```

```
    query,
```

```
    final_docs
```

```
)
```

```
return reranked[:5]
```

---

## STEP 4 — CONTEXT BUILDER

 **app/context/context\_builder.py**

```
def build_context(results):
```

```
    context = []
```

```
    for idx, r in enumerate(results):
```

```
        chunk = f"""
```

```
[Context {idx+1}]
```

```
Page: {r['metadata']['page']}
```

```
{r['document']}
```

```
"""
```

```
    context.append(chunk)
```

```
return "\n".join(context)
```

---

## ✅ STEP 5 — QWEN QUERY AGENT

 **app/agent/ask.py**

```
import ollama
```

```
from app.query.hybrid_search import (
    hybrid_search
)
```

```
from app.context.context_builder import (
    build_context
)
```

```
def ask_question(query):
```

```
    results = hybrid_search(query)
```

```
    context = build_context(results)
```

```
    prompt = f"""
```

You are an automotive engineering assistant.

Use ONLY the provided context.

Context:

{context}

Question:

```
{query}
```

```
"""
```

```
response = ollama.chat(
```

```
    model="qwen2.5",
```

```
    messages=[
```

```
        {
```

```
            "role": "user",
```

```
            "content": prompt
```

```
        }
```

```
    ]
```

```
)
```

```
return response["message"]["content"]
```

---

## ✅ STEP 6 — TEST AGENT

 **tests/test\_agent.py**

```
from app.agent.ask import (
```

```
    ask_question
```

```
)
```

```
response = ask_question(
```

```
    "What is the maximum front pad temperature?"
```

```
)
```

```
print("\n=== RESPONSE ===\n")
```

```
print(response)
```



---

## ✅ EXPECTED RESULT

The maximum front pad peak temperature observed:

CDT : 163 deg C

GHD : 179 deg C

HSD : 188 deg C

---

## ✅ WHAT DAY 9 ACHIEVES

✅ reranking

✅ context assembly

✅ engineering QA

✅ duplicate removal

✅ cleaner retrieval

✅ better answers

✅ local engineering copilot

---

## ✅ YOUR SYSTEM NOW

PDF

↓

Extraction

↓

Normalization

↓

Filtering

↓

Embeddings

↓

ChromaDB



Hybrid Search



Reranking 



Context Builder 



Qwen2.5 

---

 **YOU ARE NOW HERE**

 DAY 9 COMPLETE

Next:

DAY 10

→ metadata-aware analytics

→ structured querying

→ engineering calculations

→ DuckDB integration